

Data Science Fundamentals & Customer Credit Risk

Conceptual Foundation (Part A) • Final Data Preparation Deliverable (Part H)

Prepared by: Priya Savaliya
Data Science & AI/ML

Part A: Conceptual Foundation

1. Short Notes

1.1 What is Data Analysis?

Data Analysis is the process of inspecting, cleaning, transforming, and modeling data to discover useful information, draw meaningful conclusions, and support decision-making. It converts raw, unstructured data into insights that businesses and researchers can act on.

A typical data analysis workflow includes:

- Data Collection – gathering data from files, databases, APIs, or sensors.
- Data Cleaning – handling missing values, duplicates, and inconsistent formats.
- Exploratory Data Analysis (EDA) – summarizing data using statistics and visualizations.
- Modeling / Analysis – applying statistical or machine learning techniques.
- Interpretation & Reporting – presenting findings clearly to stakeholders.

1.2 How to Plan a Data Science Project

Planning a data science project follows a structured lifecycle (similar to CRISP-DM) so that technical work stays aligned with the business goal:

- Business Understanding – define the problem, objective, and success criteria.
- Data Understanding – identify data sources, collect sample data, assess quality.
- Data Preparation – clean, merge, transform, and engineer features.
- Modeling – select algorithms, train, and tune models.
- Evaluation – validate results against business objectives, not just accuracy metrics.
- Deployment – integrate the model into a real workflow or product.
- Monitoring & Maintenance – track performance and retrain as data drifts over time.

1.3 How to Frame a Machine Learning Problem

Framing an ML problem means translating a business question into a well-defined, solvable modeling task. Key steps include:

- Define the objective – what decision or prediction is needed?
- Choose the problem type – classification, regression, clustering, or ranking.
- Identify the target variable and the features that could predict it.
- Decide the evaluation metric (accuracy, F1-score, RMSE, AUC, etc.) based on business impact.
- Check data availability, quality, and any label/leakage issues.

- Establish a simple baseline before building complex models.

2. Tensors – In-Depth Explanation

A Tensor is a generalized n-dimensional array — a single data structure that extends the idea of scalars (0-D), vectors (1-D), and matrices (2-D) into higher dimensions (3-D, 4-D, and beyond). Tensors are the core data structure used across NumPy, TensorFlow, and PyTorch to store and manipulate numbers efficiently — from a single value to an entire batch of images.

Every tensor has three key properties:

- Rank / ndim – the number of dimensions (axes) the tensor has.
- Shape – a tuple describing the size along each axis.
- Dtype – the data type of the elements (int32, float32, etc.).

Rank	Name	Example	Shape
0	Scalar	5	()
1	Vector	[1, 2, 3]	(3,)
2	Matrix	[[1,2],[3,4]]	(2, 2)
3	3-D Tensor	RGB image (H, W, C)	(28, 28, 3)
4	4-D Tensor	Batch of images (N, H, W, C)	(32, 28, 28, 3)

Table 1: Tensor rank hierarchy from scalar to 4-D tensor

NumPy Examples

Creating tensors of different ranks:

```
import numpy as np

# 0-D Tensor (Scalar)
scalar = np.array(7)
print(scalar.ndim, scalar.shape)    # 0  ()

# 1-D Tensor (Vector)
vector = np.array([1, 2, 3, 4])
print(vector.ndim, vector.shape)    # 1  (4,)

# 2-D Tensor (Matrix)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix.ndim, matrix.shape)    # 2  (2, 3)

# 3-D Tensor
tensor_3d = np.random.rand(2, 3, 4)
print(tensor_3d.ndim, tensor_3d.shape) # 3  (2, 3, 4)
```

Reshaping and changing data type:

```
# Reshape a 3-D tensor into 2-D
reshaped = tensor_3d.reshape(4, 6)
print(reshaped.shape)              # (4, 6)

# Cast to a different dtype
float_tensor = vector.astype(np.float32)
print(float_tensor.dtype)          # float32
```

Broadcasting — NumPy automatically expands smaller tensors to match shapes during operations:

```
a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
b = np.array([10, 20, 30])          # shape (3,)

print(a + b)
# [[11 22 33]
#  [14 25 36]] -> b is broadcast across every row of a
```

This ability to represent and operate on n-dimensional data efficiently is exactly why tensors — not plain Python lists — are the backbone of every ML/DL pipeline, from a single feature row to a full batch of training data.

Part H: Final Deliverable

This section summarizes the end-to-end data preparation pipeline carried out on the Customer Credit Risk dataset (project: “Holistic Data Preparer”), which merges four sources — a transactions CSV, customer metadata JSON, loan repayment history from SQL, and regional economic indicators from a dummy API — into a single modeling-ready dataset.

Customer Credit Risk — Data Preparation Workflow

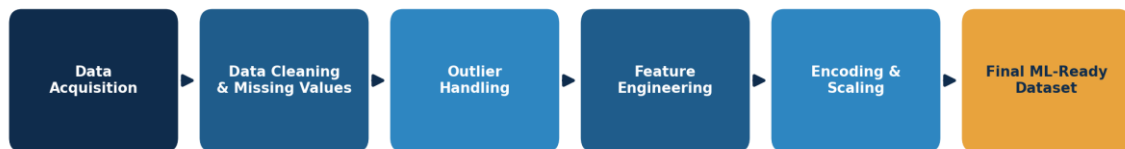


Figure 1: End-to-end data preparation workflow

14. Final Cleaned and Transformed Dataset

Merging all four sources produced a base dataset of 1,008 rows × 18 columns. After resolving missing values, treating outliers without dropping rows (Winsorization), encoding categorical variables, scaling numeric features, and constructing new ratio/date features, the dataset is fully numeric, free of nulls, and ready for ML modeling with `default_flag` as the target variable.

Stage	Rows	Columns	Notes
Raw merge (CSV + JSON + SQL + API)	1,008	18	4 sources joined on customer_id / region
After missing-value treatment	1,008	18	0 nulls remaining (KNN / MICE / Mode)
After outlier treatment (Winsorize)	1,008	18	No rows dropped, extreme values capped
After encoding (One-Hot + Ordinal + Label)	1,008	~26	Categorical columns expanded via ColumnTransformer
After feature construction	1,008	~29	+3 engineered ratio features

Table 2: Dataset shape at each pipeline stage

15. Report Summary

15.1 Missing Value Strategies Used and Their Effectiveness

Missing Values by Column (n = 1008)

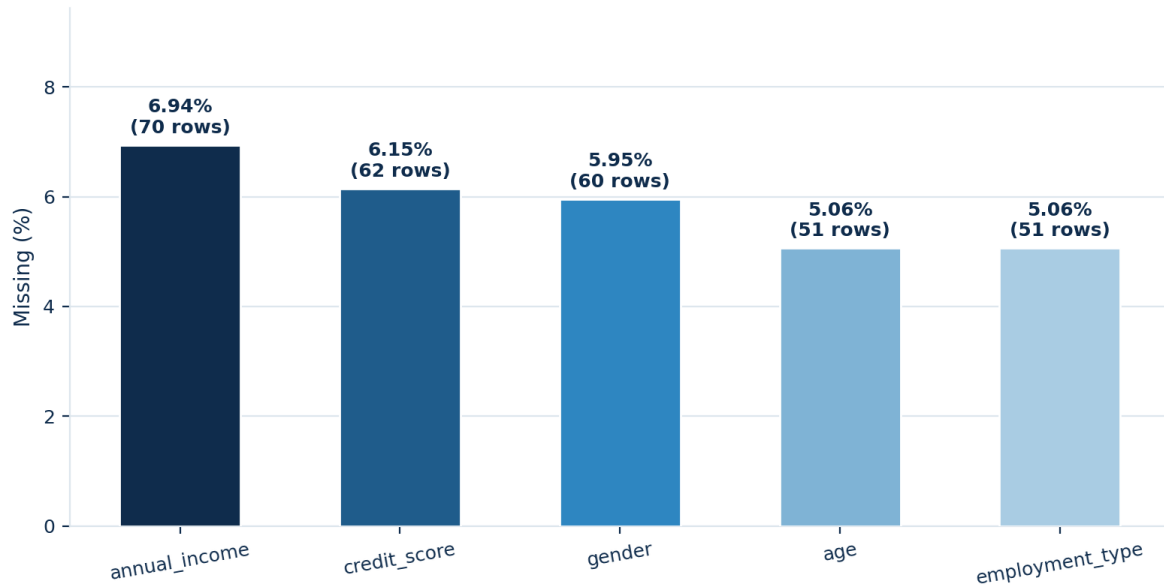


Figure 2: Missing values by column (out of 1,008 rows)

Column	Missing	Strategy Applied	Effectiveness
annual_income	70 (6.94%)	KNN Imputer (k=5)	0 missing left; preserves relation with loan_amount & credit_score
credit_score	62 (6.15%)	KNN Imputer / MICE	0 missing left; MICE models inter-feature correlation
age	51 (5.06%)	Simple Imputer (median)	0 missing left; robust to skew
gender	60 (5.95%)	Most-Frequent / Mode	0 missing left; safe for low-cardinality category
employment_type	51 (5.06%)	Simple Imputer (most frequent)	0 missing left

Table 3: Missing value strategy per column

Complete Case Analysis (row deletion) was also tested but rejected — it dropped 262 of 1,008 rows (26% data loss), which is too costly given the dataset size. Missing Indicator + Random Sample Imputation was used specifically on annual_income to flag missingness while preserving the original distribution shape.

15.2 Outlier Handling Results

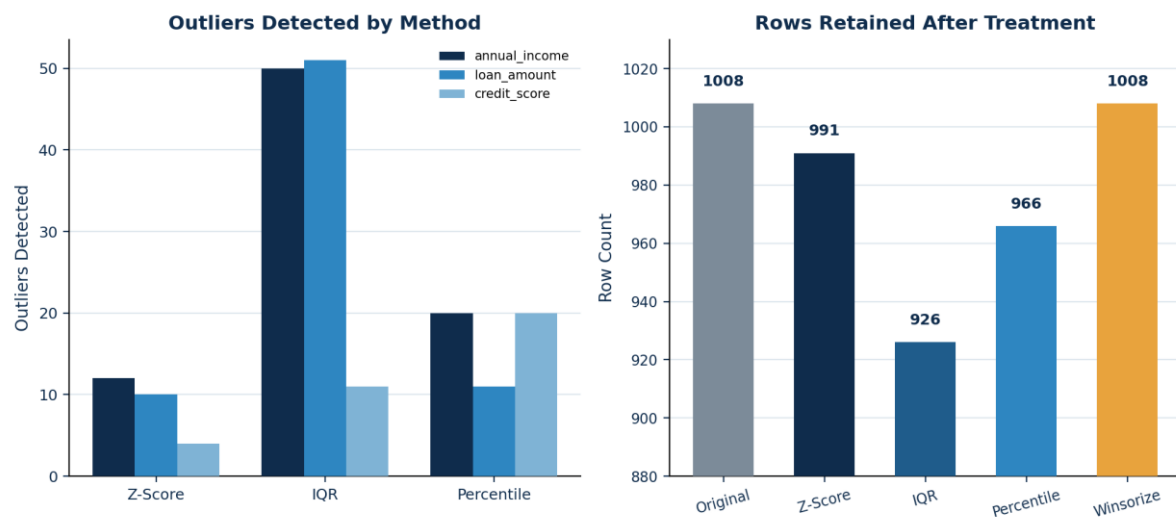


Figure 3: Outliers detected per method and rows retained after treatment

Method	annual_income	loan_amount	credit_score	Rows Retained
Z-Score ($ z > 3$)	12	10	4	991 (–1.7%)
IQR ($1.5 \times \text{IQR}$)	50	51	11	926 (–8.1%)
Percentile (1st–99th)	20	11	20	966 (–4.2%)
Winsorization (capping)	capped	capped	capped	1,008 (0% loss)

Table 4: Outlier counts by detection method

Winsorization was selected as the final treatment — it caps extreme values at the 1st/99th percentile instead of deleting rows, so the full 1,008-row sample is preserved while skew and standard deviation are reduced (e.g. annual_income minimum moved from ₹1,20,484 to ₹2,01,980 with std virtually unchanged), which matters given the dataset's already-limited size.

15.3 Encoding Methods Applied to Categorical / Numerical Variables

Variable(s)	Method	Detail
education_level	Ordinal Encoding	Primary(0) < Secondary(1) < Graduate(2) < Post-Graduate(3)
gender	Label Encoding	Female=0, Male=1, Other=2, Unknown=3
region, loan_purpose	One-Hot Encoding	18 → 25 columns (4 region + 5 loan-purpose dummies)
annual_income	Equal-width Binning	income_group: Low / Medium / High / Very High
credit_score	Binarization (>700)	high_credit_flag: 660 below vs 348 at/above
transaction_count	Quantile Binning (4)	Low 294 / Medium 272 / High 215 / Very High 227
transaction_count	K-Means Binning (4 clusters)	Data-driven alternative to fixed-width bins
join_date	Date decomposition	join_year, join_month, join_day, join_weekday

Table 5: Encoding methods by variable

15.4 Scaling / Transformations Applied and Why

Method	Behaviour	Why Used
Standardization	mean = 0, std = 1	Suits Logistic Regression / SVM; centers skewed income & loan values
Normalization (L2)	Row vector scaled to unit norm	Useful when direction of the feature vector matters more than scale
Min-Max Scaling	Rescales to [0, 1]	Needed for neural nets / gradient-based models sensitive to range
MaxAbs Scaling	Scales by max absolute value	Preserves sign & zero entries, safe for sparse data
Robust Scaling	Uses median & IQR	Least sensitive to remaining outliers — best fit for this skewed financial data
Box-Cox Transform	Power transform (positive values)	Corrects strong right-skew in annual_income
Yeo-Johnson Transform	Power transform (handles zero/negative)	Normalizes loan_amount distribution

Table 6: Scaling and transformation methods compared

Robust Scaling combined with Yeo-Johnson / Box-Cox transformation was chosen for the final pipeline, since the financial features (income, loan amount) remain right-skewed even after outlier capping.

15.5 Newly Engineered Features and Their Usefulness

Feature	Formula	Usefulness
debt_to_income_ratio	loan_amount / annual_income	Direct credit-risk signal — higher ratio implies higher default risk
average_monthly_transactions	transaction_count / 6	Normalizes activity over the observation window for fair comparison
spending_to_income_ratio	spending_ratio / 100	Converts % to a 0–1 ratio, consistent scale with other engineered features
join_year / month / day / weekday	Extracted from join_date	Enables tenure and seasonality-based analysis
high_credit_flag	credit_score > 700	Quick binary segmentation for reporting
income_group / transaction_quantile / transaction_cluster	Binned / clustered	Interpretable customer segments for business stakeholders

Table 7: Engineered features and their purpose

15.6 Final Dataset Shape and Readiness for ML Modeling

After the full pipeline — KNN/MICE/mode imputation, Winsorization for outliers, one-hot + ordinal + label encoding, and robust scaling with Yeo-Johnson/Box-Cox transformation — the working dataset stands at approximately 1,008 rows × 29 columns: 26 processed base features (from the ColumnTransformer) plus 3 constructed ratio features, with the option to add date-part and segment columns for reporting.

- Zero missing values across all base columns.

- No rows dropped — full 1,008-record sample preserved.
- All categorical variables converted to numeric form (ordinal / label / one-hot).
- Numeric features scaled and de-skewed, ready for distance- and gradient-based algorithms.
- Engineered ratio features add direct domain signal for credit-risk prediction.

The dataset is clean, fully numeric, leak-checked, and structurally ready for training classification models against the `default_flag` target.